

Mobile App Error Handling and Troubleshooting

Mobile identity, network, payment and release troubleshooting.



This document explains its subject first in plain language, then in engineering detail. It is grounded in the Craves repository snapshot and deliberately separates current implementation, legacy/reference code, milestone foundations and repository gaps so readers do not mistake aspiration for proof.

Version 1.0 | Evidence date: 14 August 2026 | Repository: [rmorampudi09-arch/Craves-Build-platform](#) | Snapshot commit: [3225f7fa531a6b07185fb5e034f7930f8bd8b571](#)

mobile evidence boundary - plain-language purpose

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile evidence boundary - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile evidence boundary - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile evidence boundary - errors and edge cases

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

mobile evidence boundary - deployment, security and operational validation

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Entra External ID - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

Entra External ID - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Entra External ID - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Entra External ID - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

Entra External ID - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE - plain-language purpose

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE - errors and edge cases

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

PKCE - deployment, security and operational validation

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

MFA - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

MFA - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

MFA - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

MFA - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

MFA - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

interactive sign-in - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

interactive sign-in - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

interactive sign-in - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

interactive sign-in - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

interactive sign-in - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context.
2. Client/BFF validates local prerequisites and sends an API request.
3. APIM and the owning service enforce identity, role, ownership and business-state rules.
4. The domain service reads/writes authoritative state and calls providers only through defined boundaries.
5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier.
2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets.
3. Verify caller role/ownership and current authoritative state before retrying.
4. Check owning service health, then its provider/dependency after local/configuration causes are excluded.
5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

refresh token secure storage - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

refresh token secure storage - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

refresh token secure storage - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

refresh token secure storage - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

refresh token secure storage - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

access token memory - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

access token memory - technical architecture

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

access token memory - end-to-end flow

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

access token memory - errors and edge cases

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

access token memory - deployment, security and operational validation

Plain-English explanation

Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone.

How it works technically

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The auth boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM bearer calls - plain-language purpose

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM bearer calls - technical architecture

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM bearer calls - end-to-end flow

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM bearer calls - errors and edge cases

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

APIM bearer calls - deployment, security and operational validation

Plain-English explanation

Azure API Management is the intended policy and routing front door between clients/BFFs and backend services.

How it works technically

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The apim boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: infra/apim/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

401 refresh/retry - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

401 refresh/retry - technical architecture

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

401 refresh/retry - end-to-end flow

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

401 refresh/retry - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

401 refresh/retry - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

logout - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and [apps/customer-web-next/docs/signalr-ai-concierge.md](#) describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: [apps/customer-web-next/docs/signalr-ai-concierge.md](#); commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

logout - technical architecture

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

logout - end-to-end flow

Plain-English explanation

Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot.

How it works technically

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The mobile boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Milestone foundation. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

logout - errors and edge cases

Plain-English explanation

A useful failure model separates user input/state conflicts, identity/permission failures, dependency outages, data faults and deployment/configuration incidents.

How it works technically

order-service documents Problem Details fields and codes including ORDER_NOT_FOUND, ORDER_IDEMPOTENCY_CONFLICT, INVALID_STATUS, CANCELLATION_NOT_ALLOWED, REFUND_FAILED, UNAUTHORIZED, FORBIDDEN, BACKEND_CHANGE_DISABLED and BACKEND_UNAVAILABLE. Diagnostics should start from request/correlation evidence.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The errors boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: services/order-service/README.md; services/notification-service/README.md; apps/customer-web-next/README.md. Snapshot: main @ 3225f7fa531a (14 August 2026).

logout - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

environment config - plain-language purpose

Plain-English explanation

Craves includes a governed semantic meal concierge for assisted discovery rather than an authority that can bypass commerce rules.

How it works technically

Recent main history and apps/customer-web-next/docs/signalr-ai-concierge.md describe SignalR semantic concierge work. Recommendations still rely on governed product data and ordinary catalog, quote, authorization and order paths for transactions.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The ai boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/docs/signalr-ai-concierge.md; commit 71e795b. Snapshot: main @ 3225f7fa531a (14 August 2026).

environment config - technical architecture

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

environment config - end-to-end flow

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

environment config - errors and edge cases

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).

environment config - deployment, security and operational validation

Plain-English explanation

The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls.

How it works technically

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers.

Flow described in words

1. Actor starts the capability with the appropriate identity/context. 2. Client/BFF validates local prerequisites and sends an API request. 3. APIM and the owning service enforce identity, role, ownership and business-state rules. 4. The domain service reads/writes authoritative state and calls providers only through defined boundaries. 5. The caller receives authoritative result or a traceable error; support uses request/correlation evidence rather than secrets.

Errors, edge cases and controls

- Stale UI/cache must not override server state.
- UI visibility is not authorization evidence.
- Retries of money/state-changing operations require idempotency awareness.
- Provider/configuration outages should preserve core domain consistency where safe.
- Missing repository proof is reported as a gap rather than converted into a production claim.

Diagnostic and validation steps

1. Capture time, environment, operation, expected/actual result and a user-safe identifier. 2. Capture HTTP status, stable error code and request/correlation ID; never copy OTPs, passwords, access tokens or provider secrets. 3. Verify caller role/ownership and current authoritative state before retrying. 4. Check owning service health, then its provider/dependency after local/configuration causes are excluded. 5. If a release is implicated, compare artifact/config marker with prior known-good revision and validate rollback compatibility.

Data, security and deployment impact

The devops boundary owns its authoritative state; clients request or display changes but do not become the source of truth. Secrets and unnecessary personal data stay outside logs and PDFs. Releases must keep API, policy, schema and artifact compatible, with health/readiness evidence and rollback/backout appropriate to the change.

Evidence status

Implemented on main. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Snapshot: main @ 3225f7fa531a (14 August 2026).